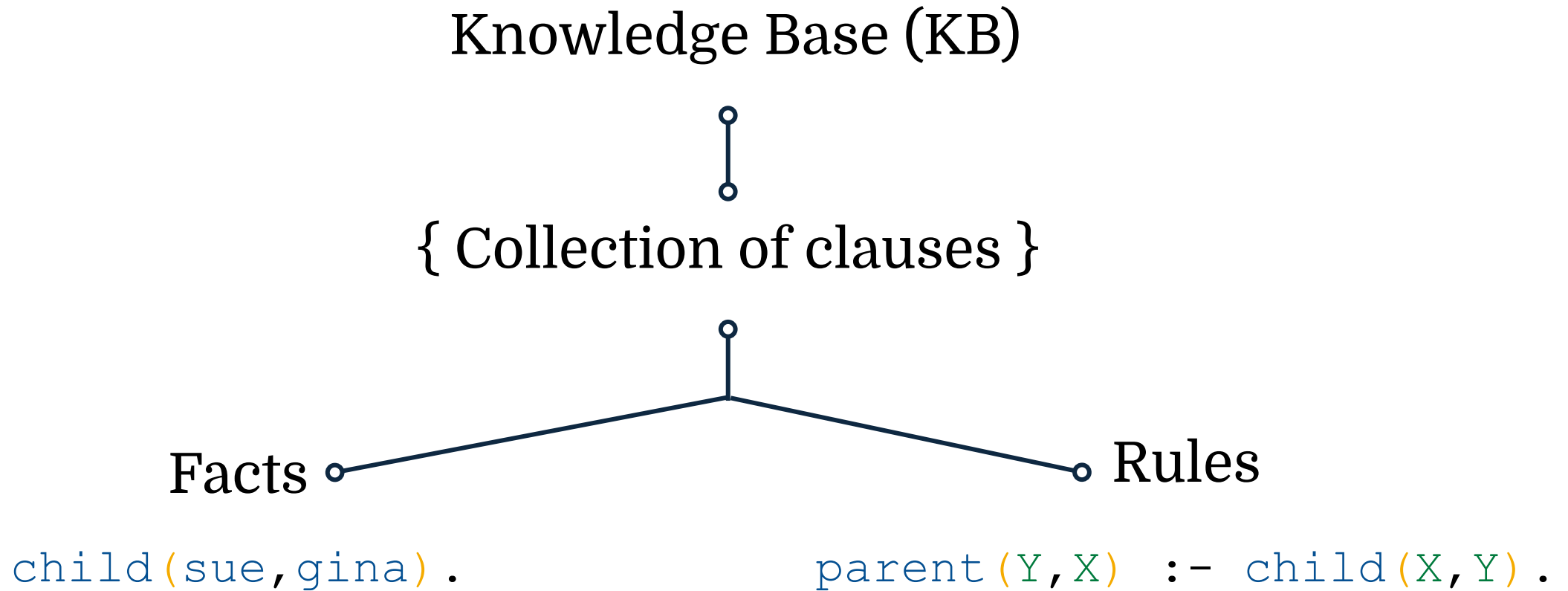


Prolog – Unification, Search and Datatypes

Multi-Agent Systems
Practical 2

Recap



Recap - atoms

Also known as a **textual constant**.

- A string of characters beginning with a lowercase letter.
- Characters enclosed with **Single quotes**.
- A string of special characters.

{ Knowledge Base }

```
child(sue, gINA) .
```

```
child(sue, '•□•') .
```

```
@= ('•□•') .
```

```
female(sue) .
```

```
parent(Y, X) :- child(X, Y) .
```

Recap - Variables

A string of upper-case letters, lower-case letters, digits and underscore characters that starts with:

- An upper-case letter.
- An underscore.

{ Knowledge Base }

```
parent(Y, Var) :- child(Var, Y) .  
child(_X, Y8)  :- parent(Y8, _X) .
```

Recap - Compound terms

Also known as a **structure**.

A **functor** followed by a sequence of **arguments**.

⋮
atom

⋮
{ **atoms**, **Variables**, numbers }

```
female(sue) .                                % arity 1
parent(Y, Var) .                             % arity 2

hide(X, Z, father(father(father(sue))))      % arity 3
father(father(father(sue)))                  % arity 1
```

Unification in Prolog



Two terms t_1 and t_2 match IF AND ONLY IF:

t_1 and t_2 are identical **constants**.

If t_1 is a **variable** and t_2 is not, then t_1 is instantiated with t_2 .

```
?- =(female, female) .  
    % true
```

```
?- =(female, male) .  
    % false
```

```
?- =(X, male) .  
    % X = male
```

Unification in Prolog



If t_1 and t_2 are **compound terms**, they match if and only if:

1. They have the same outermost functor f .

$$t_1 = f(\dots, \dots) . \quad \text{and} \quad t_2 = f(\dots, \dots) .$$

2. Each **argument** in t_1 matches with every **argument** in t_2 .

$$t_1 = f(\text{mia}) . \quad \text{and} \quad t_2 = f(\text{mia}) .$$

3. The **variable** instantiations are compatible, i.e., a variable cannot be assigned different terms which are not variables.

$$t_1 = f(X, X) . \quad \text{and} \quad t_2 = f(a, b) .$$

% Fails

$$t_1 = f(X, X) . \quad \text{and} \quad t_2 = f(a, a) .$$

% X = a

Unification in Prolog



`=/2` is the built-in matching predicate

```
?- =(female, female) .  
    % true  
  
?- =(female, male) .  
    % false  
  
?- =(f(X, X), f(a, b)) .  
    % X = female
```

`\=/2` can be used to check for *terms that do not match*

```
?- \=(female, female) .  
    % false  
  
?- \=(female, male) .  
    % true  
  
?- \=(f(X, X), f(a, b)) .  
    % true
```

Search tree example



$p(a)$

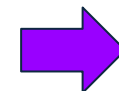
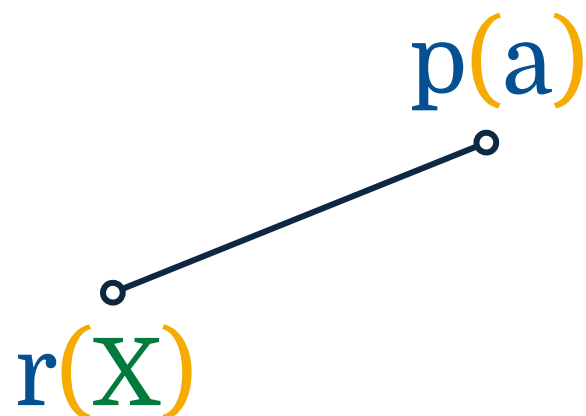
Knowledge base

```
s (a) .  
r (b) .  
p (X) :- r (X) .  
p (X) :- s (X) .  
p (X) :- t (X) .  
t (X) :- t (X) .
```

Query

```
?- p(a).
```

Search tree example



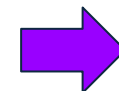
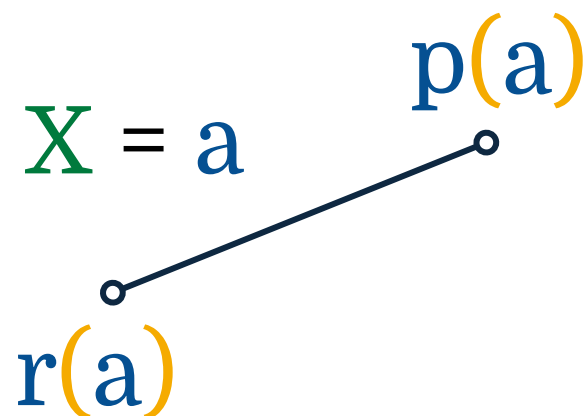
Knowledge base

```
s (a) .  
r (b) .  
p (X) :- r (X) .  
p (X) :- s (X) .  
p (X) :- t (X) .  
t (X) :- t (X) .
```

Query

```
?- p(a).
```

Search tree example



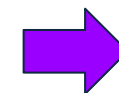
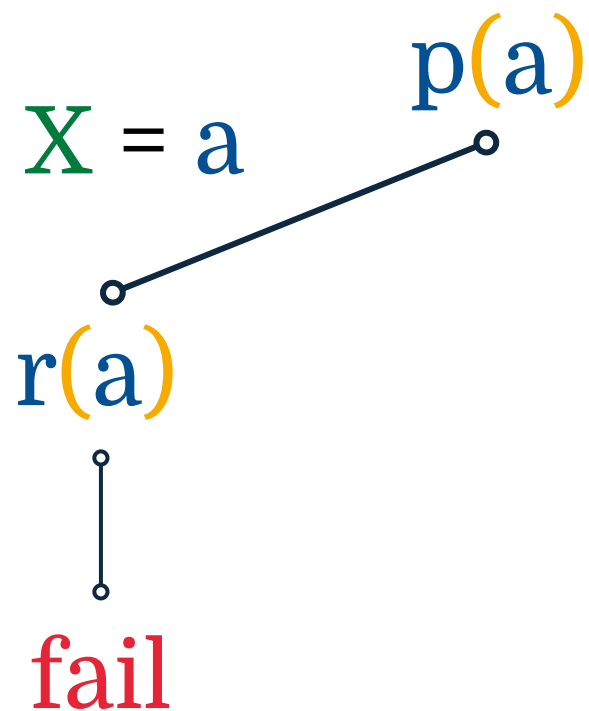
Knowledge base

```
s (a) .  
r (b) .  
p (X) :- r (X) .  
p (X) :- s (X) .  
p (X) :- t (X) .  
t (X) :- t (X) .
```

Query

```
?- p(a).
```

Search tree example



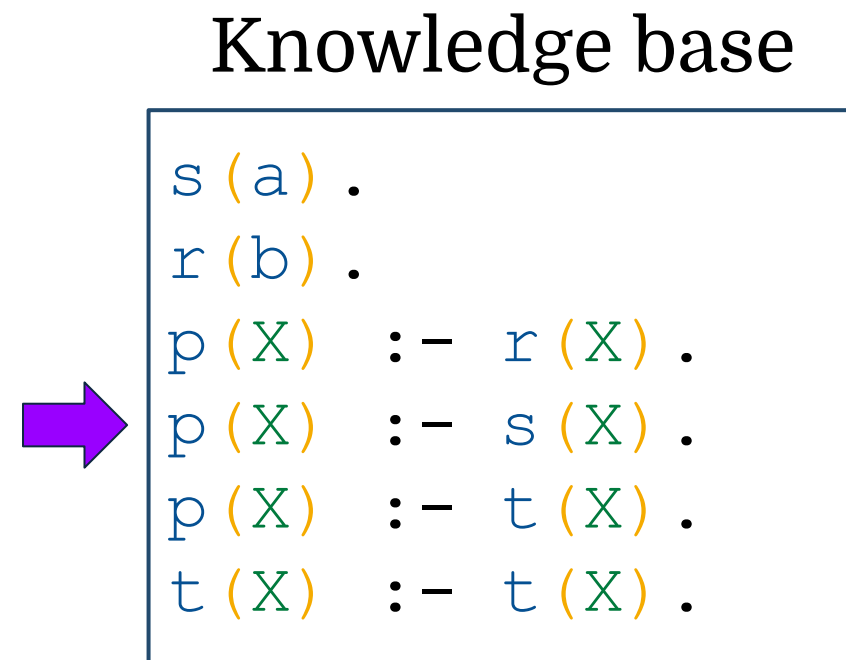
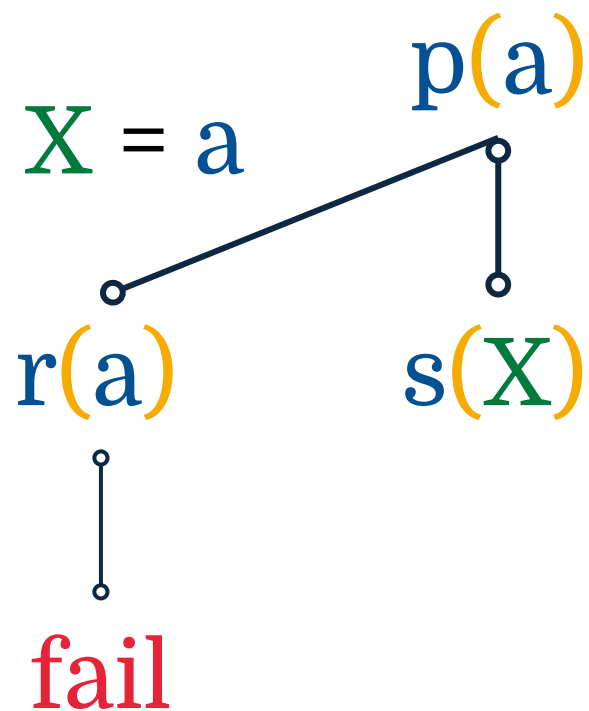
Knowledge base

```
s (a) .  
r (b) .  
p (X) :- r (X) .  
p (X) :- s (X) .  
p (X) :- t (X) .  
t (X) :- t (X) .
```

Query

```
?- p(a).
```

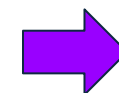
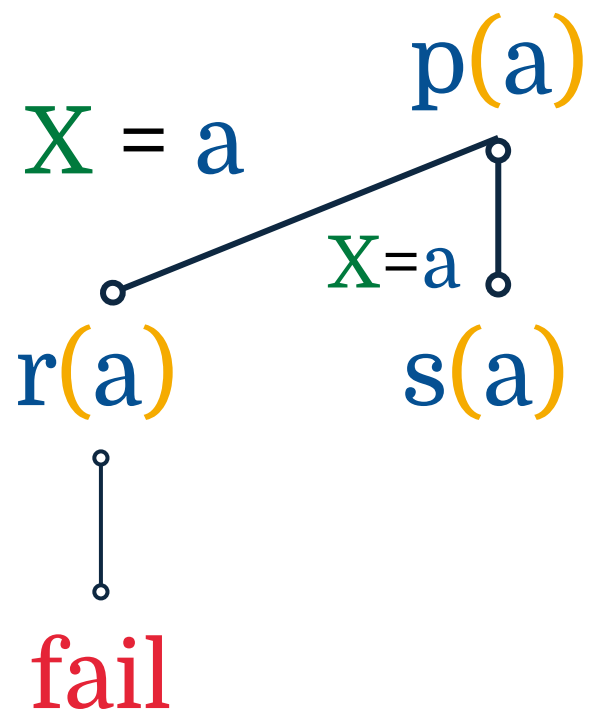
Search tree example



Query

?- $p(a)$.

Search tree example



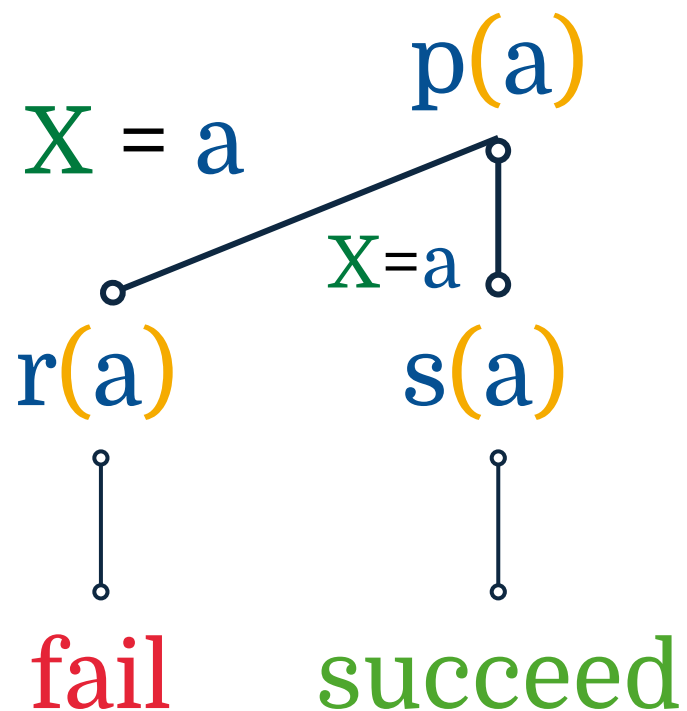
Knowledge base

```
s(a) .  
r(b) .  
p(X) :- r(X) .  
p(X) :- s(X) .  
p(X) :- t(X) .  
t(X) :- t(X) .
```

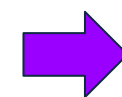
Query

```
?- p(a).
```

Search tree example



Knowledge base

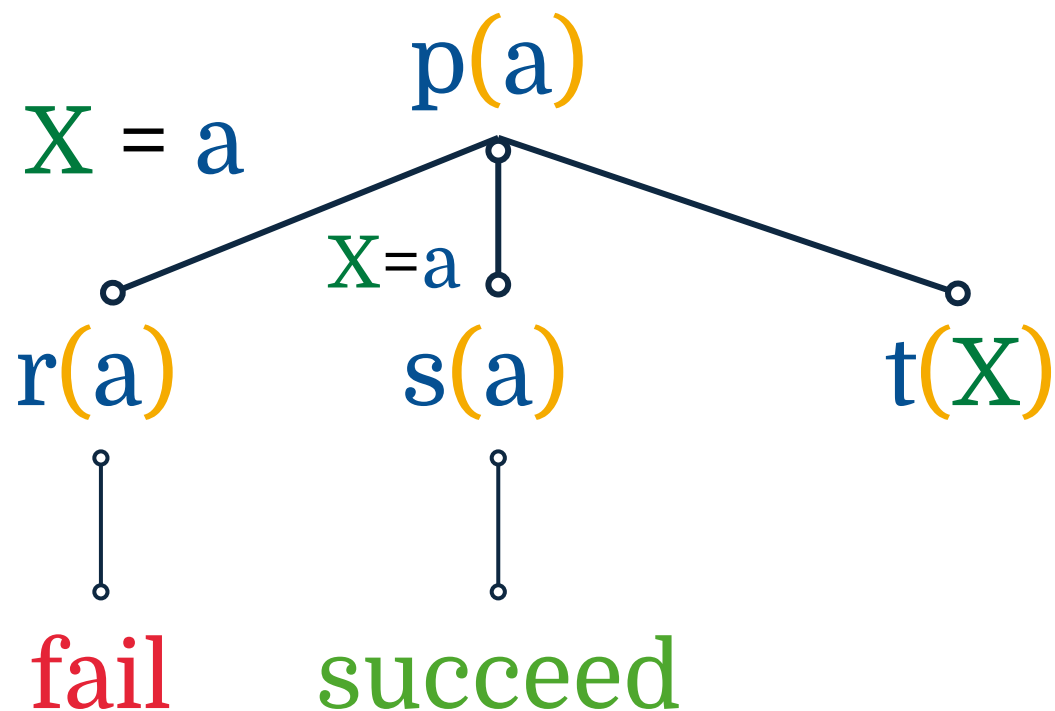


```
s (a) .  
r (b) .  
p (X) :- r (X) .  
p (X) :- s (X) .  
p (X) :- t (X) .  
t (X) :- t (X) .
```

Query

```
?- p(a).
```

Search tree example



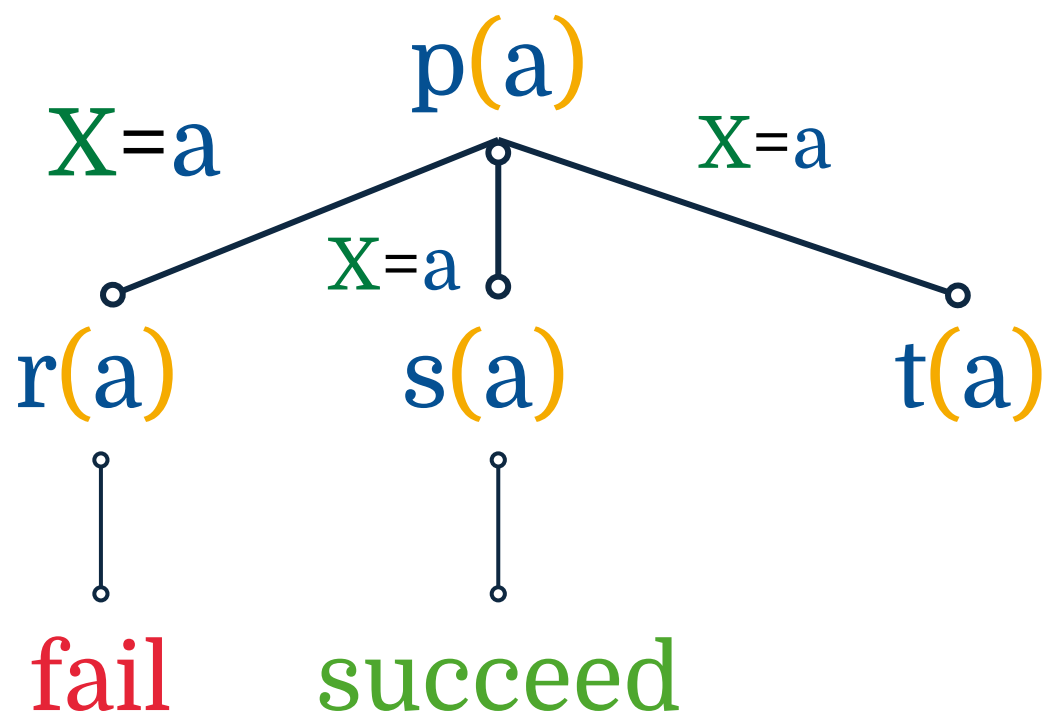
Knowledge base

```
s(a) .  
r(b) .  
p(X) :- r(X) .  
p(X) :- s(X) .  
p(X) :- t(X) .  
t(X) :- t(X) .
```

Query

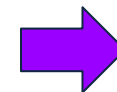
```
?- p(a).
```

Search tree example



Knowledge base

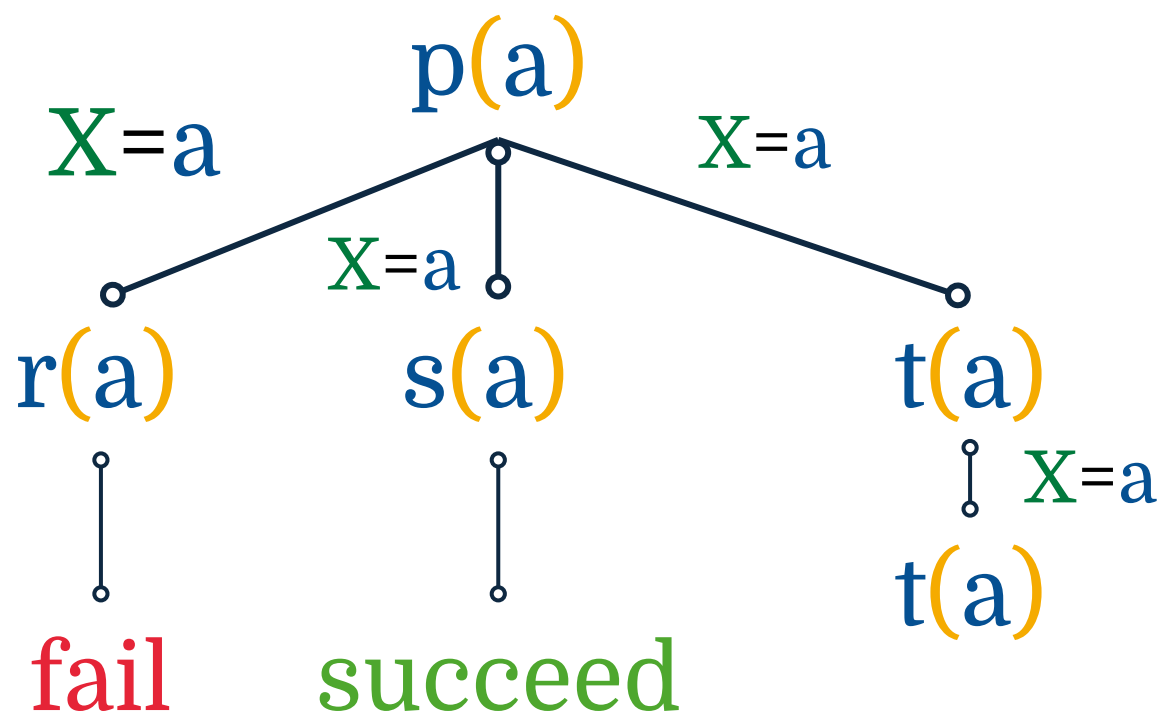
```
s (a) .  
r (b) .  
p (X) :- r (X) .  
p (X) :- s (X) .  
p (X) :- t (X) .  
t (X) :- t (X) .
```



Query

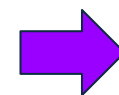
```
?- p(a).
```

Search tree example



Knowledge base

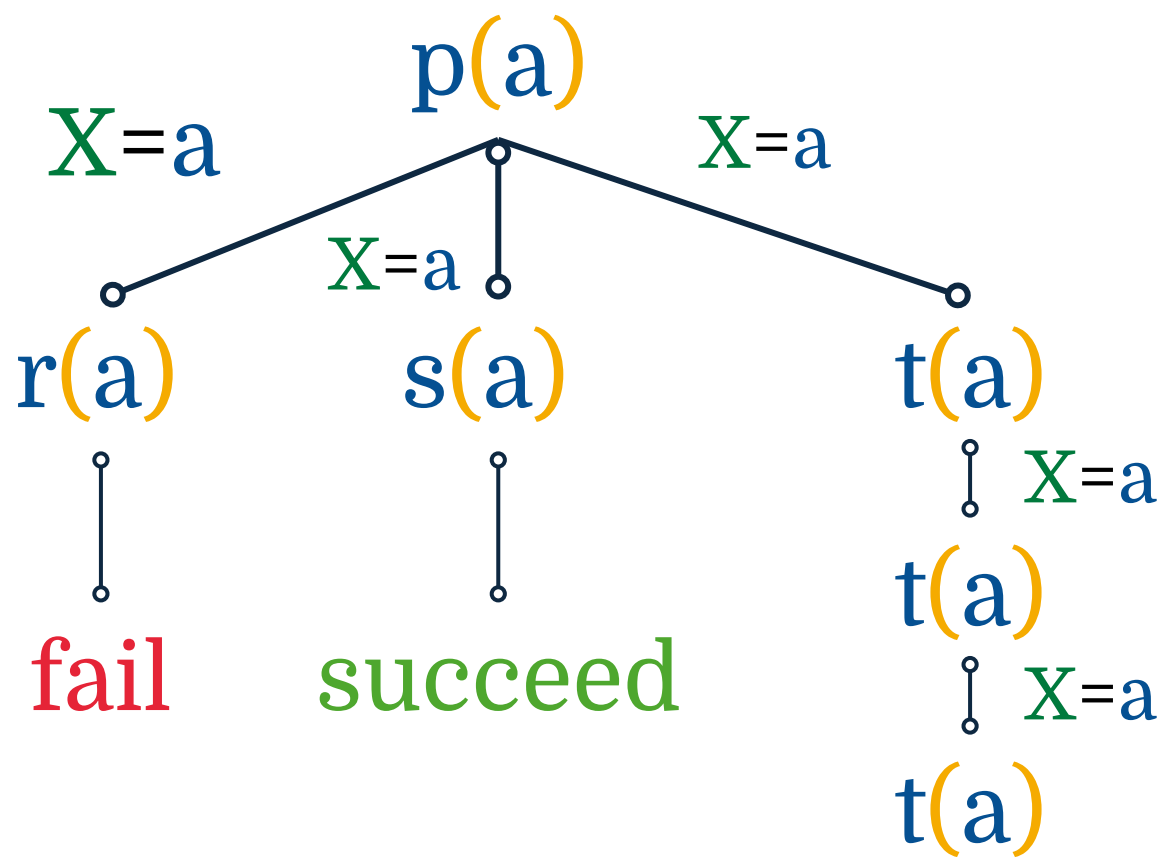
```
s(a) .  
r(b) .  
p(X) :- r(X) .  
p(X) :- s(X) .  
p(X) :- t(X) .  
t(X) :- t(X) .
```



Query

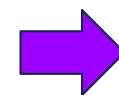
```
?- p(a).
```

Search tree example



Knowledge base

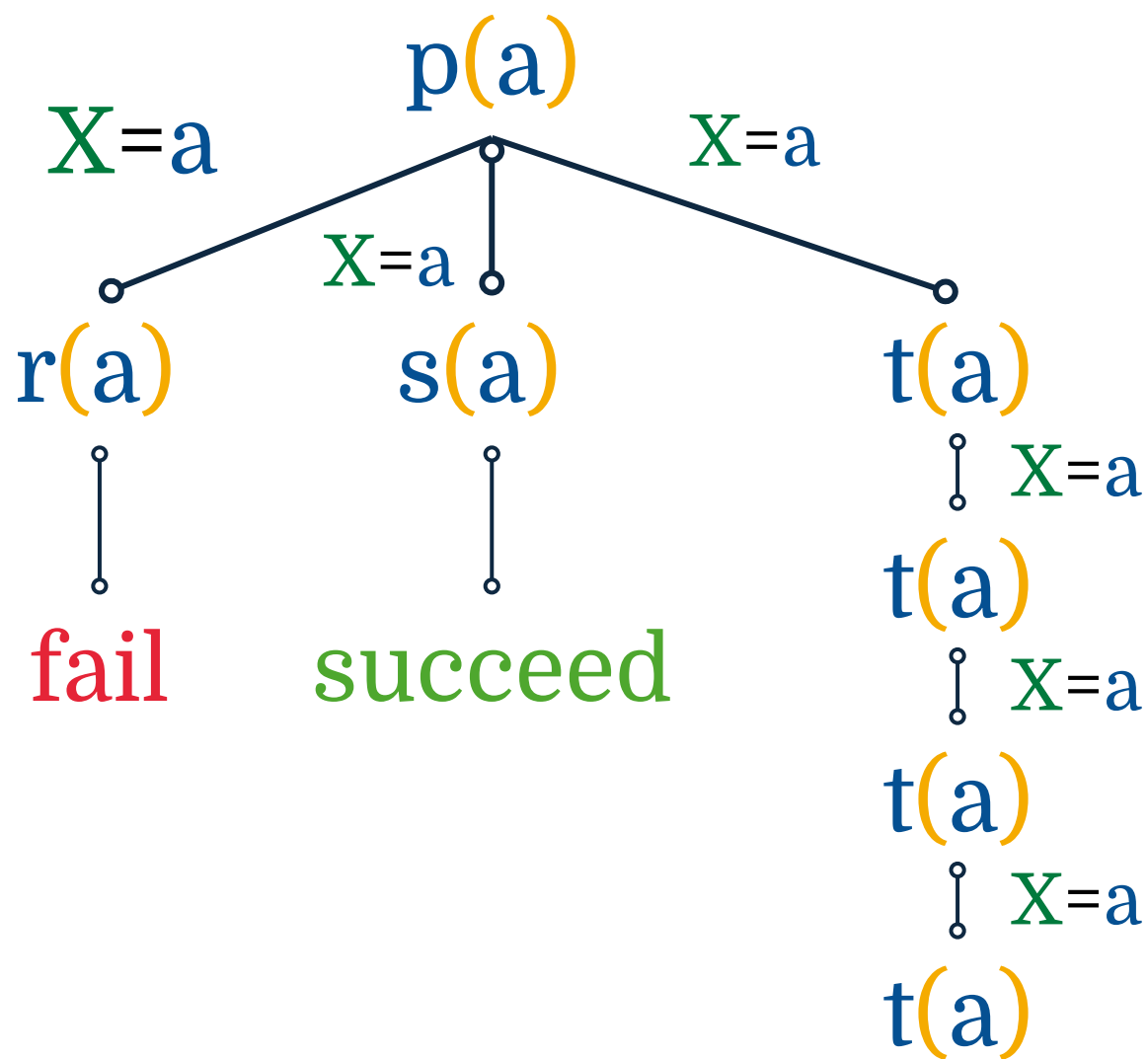
```
s (a) .  
r (b) .  
p (X) :- r (X) .  
p (X) :- s (X) .  
p (X) :- t (X) .  
t (X) :- t (X) .
```



Query

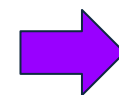
```
?- p(a).
```

Search tree example



Knowledge base

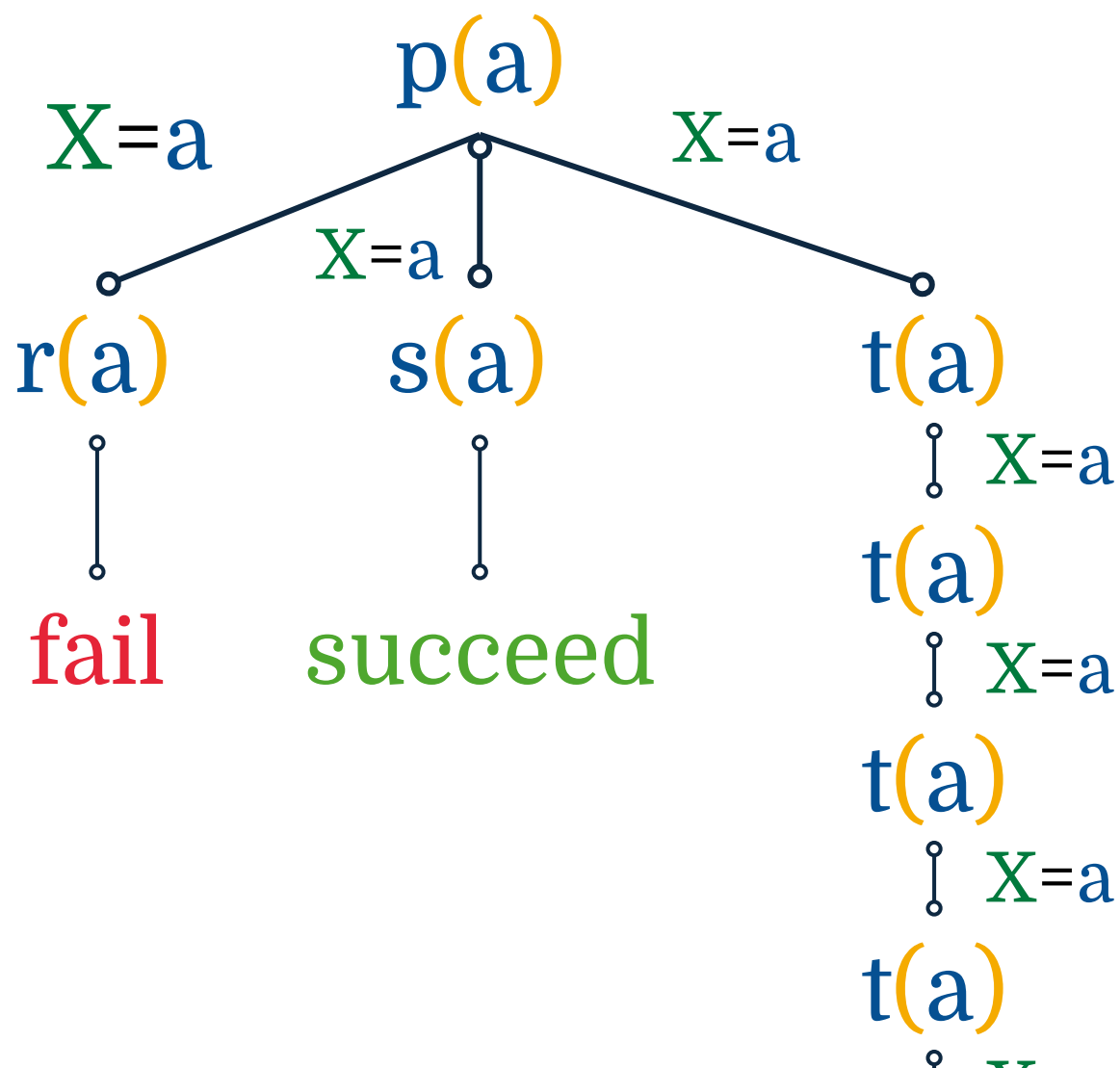
```
s (a) .  
r (b) .  
p (X) :- r (X) .  
p (X) :- s (X) .  
p (X) :- t (X) .  
t (X) :- t (X) .
```



Query

```
?- p(a).
```

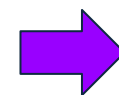
Search tree example



Knowledge base

```

s (a) .
r (b) .
p (X) :- r (X) .
p (X) :- s (X) .
p (X) :- t (X) .
t (X) :- t (X) .
    
```



Query

```

?- p(a).
    
```

Datatypes

Basic datatypes

- Atoms hero.
- Numbers 1.0.
- Variables X.
- Lists [H | T].
- Compound terms f(g(z)).
- Strings “Hello world”.

Operators

- = matching
- \+ negation
- =:= match after left and right eval
- =\= mismatch after left and right eval
- !s right eval, then unification with left

Exercise 1: Modifying a Sentence

Given the sentence "Prolog is powerful and fun", you will:

```
ex1(Result):- Original = "Prolog is powerful and fun",
```

1. Extract the substring "Prolog".
`sub_string(Original, 0, 6, _, PrologWord),`
2. Extract the substring "fun".
`sub_string(Original, 23, 3, _, FunWord),`
3. Rebuild a new sentence that says "Learning Prolog is fun!"

```
% Concatenate "Learning " with "Prolog"
```

```
string_concat("Learning ", PrologWord, Part1),
```

```
% Concatenate " is " with "fun!"
```

```
string_concat(" is ", FunWord, Part2),
```

```
string_concat(Part2, "!", Part3),
```

```
% Final concatenation to build the complete sentence
```

```
string_concat(Part1, Part3, Result).
```

You will use `string_concat/3` and `sub_string/5` to achieve these tasks. Look up the documentation online to learn more about these built-in predicates.

Exercise 2: Group Management System

Given a list of students

Students = ["Alice", "Bob", "Charlie", "Diana", "Eve", "Frank", "Grace", "Hannah"].

Use built-in list predicates: <https://www.swi-prolog.org/pldoc/man?section=lists>

- 1. Check Enrollment:** write a query that checks if "Alice" and "Grace" are both enrolled in the Students list.

```
member("Alice", Students), member("Grace", Students).  
% Expected: true
```
- 2. Count Total Students**

```
length(Students, Count).  
% Count = 8
```
- 3. Reverse the List of Students**

```
reverse(Students, ReversedStudents).  
% ReversedStudents = ["Hannah", "Grace", "Frank", "Eve", "Diana", "Charlie", "Bob", "Alice"]
```
- 4. Create Student Groups:** split the Students list into two equal groups (Group1 and Group2). Use append/3 to divide the list such that Group1 contains the first half and Group2 contains the second half.

```
append(Group1, Group2, Students), length(Group1, 4).  
% Group1 = ["Alice", "Bob", "Charlie", "Diana"]  
% Group2 = ["Eve", "Frank", "Grace", "Hannah"]
```
- 5. Find the position of "Eve".**

```
nth0(Position, Students, "Eve").  
% Position = 4
```
- 6. Remove "Charlie" from the list.**

```
select("Charlie", Students, UpdatedStudents).  
% UpdatedStudents = ["Alice", "Bob", "Diana", "Eve", "Frank", "Grace", "Hannah"]
```
- 7. Find All Students with names of length 5:** use length/2 and findall/3.

```
findall(Student, (member(Student, Students), string_length(Student, 5)), FiveLetterNames).  
% FiveLetterNames = ["Alice", "Grace"]
```
- 8. Generate Unique Pairs of Students:** use member/2 and findall/3.

```
findall([A, B], (member(A, Students), member(B, Students), A @< B), Pairs)  
.% Pairs contains all unique pairs, such as ["Alice", "Bob"], ["Alice", "Charlie"], etc.
```